ROADSIDE VEHICLE ASSISTANCE & MECHANIC BOOKING PLATFORM

A Comprehensive Project Report

Project Name: Roadside Vehicle Assistance & Mechanic Booking Platform

Project Code: RVA Pro

Organization: Unified Mentor

Developer: Varun Mittapalli

Technology Stack: Next.js 14, React, Node.js, Express.js, MongoDB

Version: 1.0 (Phase 1)

Date: July 2026

Table of Contents

Table of Contents 2
List of Figures 6
List of Tables 6
Chapter 1: Abstract 7
Chapter 2: Introduction 8
2.1 Overview 8
2.2 Problem Statement 8
2.3 Objectives 8
2.3.1 Primary Objectives 8
2.3.2 Secondary Objectives 8
2.4 Scope of the Project 9
2.4.1 In Scope (Phase 1) 9
2.4.2 Out of Scope (Phase 1) 9
2.5 Project Significance 9
Chapter 3: Literature Survey 10
3.1 Existing Systems 10
3.1.1 AAA (American Automobile Association) 10
3.1.2 Mechanic On-Demand Apps 10
3.1.3 Ride-Hailing Model Inspiration 10
3.2 Technology Review 10
3.2.1 Next.js and React Ecosystem 10
3.2.2 Node.js and Express.js 10
3.2.3 MongoDB and Geospatial Capabilities 11
3.2.4 JWT Authentication 11
3.3 Research Gap 11
Chapter 4: System Analysis 11
4.1 Requirement Analysis 11
4.1.1 Functional Requirements — Vehicle Owner 11
4.1.2 Functional Requirements — Mechanic / Towing Partner 11
4.1.3 Functional Requirements — Admin 12
4.1.4 Non-Functional Requirements 12

4.2 Feasibility Study 12
4.2.1 Technical Feasibility 12
4.2.2 Operational Feasibility 13
4.2.3 Economic Feasibility 13
Chapter 5: System Design 14
5.1 System Architecture 14
5.2 Database Design 14
5.2.1 Entity-Relationship Diagram 14
5.2.2 User Collection Schema 15
5.2.3 Vehicle Collection Schema 15
5.2.4 ServiceRequest Collection Schema 15
5.2.5 Database Indexes 16
5.3 Use Case Diagram 16
5.4 Service Request Status Flow 17
5.5 Data Flow Diagrams 17
5.5.1 Level 0 (Context Diagram) 17
5.5.2 Level 1 (Detailed DFD) 18
5.6 API Design 18
5.7 Frontend Component Architecture 19
5.8 Authentication Flow 20
5.9 Frontend Route Structure 21
Chapter 6: Implementation 21
6.1 Technology Stack 21
6.1.1 Frontend Technologies 21
6.1.2 Backend Technologies 22
6.2 Project Structure 22
6.3 Key Implementation Details 23
6.3.1 Haversine Distance Calculation 23
6.3.2 ETA Estimation Algorithm 24
6.3.3 JWT Authentication Middleware 24
6.3.4 Service Request Lifecycle 25
6.3.5 Nearby Mechanic Discovery 25

6.3.6 Rating System 25
6.4 Environment Configuration 25
6.4.1 Backend Environment Variables (.env) 25
6.4.2 Frontend Environment Variables (.env.local) 26
Chapter 7: Testing 26
7.1 Testing Strategy 26
7.2 Test Cases — Authentication Module 26
7.3 Test Cases — Service Request Module 26
7.4 Test Cases — Admin Module 27
7.5 Demo Test Accounts 27
Chapter 8: Screenshots & Output 28
8.1 Landing Page (Home) 28
8.2 Login Page 28
8.3 Registration Page 29
8.4 User Dashboard 30
8.5 Vehicle Management 30
8.6 Nearby Mechanics View 31
8.7 Live Service Tracking 31
8.8 Service History 32
8.9 Mechanic Dashboard 32
8.10 Mechanic Pending Requests 33
8.11 Mechanic Earnings 33
8.12 Admin KPI Dashboard 34
8.13 Admin Partner Verification 34
8.14 Admin Request Monitoring 35
Chapter 9: Deployment 35
9.1 Deployment Architecture 35
9.2 Deployment Steps 35
9.2.1 Backend Deployment (Render) 35
9.2.2 Frontend Deployment (Vercel) 36
9.2.3 Database Setup (MongoDB Atlas) 36
9.3 Deployment Checklist 36

Chapter 10: Conclusion and Future Scope 37
10.1 Conclusion 37
10.2 Key Performance Indicators 37
10.3 Future Enhancements (Phase 2+) 37
10.4 Lessons Learned 38
Chapter 11: Bibliography 39
Appendix A: Setup and Installation Guide 40
A.1 Prerequisites 40
A.2 Backend Setup 40
A.3 Frontend Setup 40
A.4 Accessing the Application 41

List of Figures

Figure No.	Description
Figure 1	System Architecture Diagram
Figure 2	Entity-Relationship (ER) Diagram
Figure 3	Use Case Diagram
Figure 4	Service Request Status Flow Diagram
Figure 5	Data Flow Diagram (Level 0)
Figure 6	Data Flow Diagram (Level 1)
Figure 7	Frontend Component Hierarchy
Figure 8	Authentication Flow Sequence Diagram
Figure 9	Deployment Architecture

List of Tables

Table No.	Description
Table 1	Technology Stack Comparison
Table 2	Functional Requirements — Vehicle Owner
Table 3	Functional Requirements — Mechanic / Towing Partner
Table 4	Functional Requirements — Admin
Table 5	Non-Functional Requirements
Table 6	API Endpoints Summary
Table 7	Database Schema — User Collection
Table 8	Database Schema — Vehicle Collection
Table 9	Database Schema — ServiceRequest Collection
Table 10	Frontend Route Structure
Table 11	Backend Dependencies
Table 12	Frontend Dependencies
Table 13	Test Cases — Authentication Module
Table 14	Test Cases — Service Request Module
Table 15	Test Cases — Admin Module
Table 16	Demo Accounts
Table 17	KPI Metrics

Chapter 1: Abstract

The Roadside Vehicle Assistance & Mechanic Booking Platform (Roadside AAA) is a full-stack web application that provides real-time, on-demand roadside assistance by connecting vehicle owners with verified nearby mechanics and towing service providers. The platform addresses critical challenges faced by vehicle owners during breakdowns, including difficulty finding reliable mechanics, unclear pricing structures, delayed support, lack of real-time tracking, and over-dependence on manual phone calls.

Built using a modern MERN-like stack comprising Next.js 14 with TypeScript on the frontend and Node.js with Express.js on the backend, the application leverages MongoDB with GeoJSON support for geospatial queries to locate nearby service providers. The system implements JWT-based authentication, role-based access control (RBAC) for three distinct user roles (vehicle owner, mechanic/towing partner, and administrator), and a comprehensive service request lifecycle management system with real-time status tracking.

Key features include vehicle management, service request creation with six service types (breakdown repair, towing, battery jump-start, flat tire repair, fuel delivery, and lockout assistance), nearby mechanic discovery using the Haversine formula for distance calculation, estimated time of arrival (ETA) computation, transparent pricing, live service tracking via polling, service history with ratings and feedback, mechanic earnings dashboard, and a comprehensive admin panel with KPI monitoring, partner verification, and dispute resolution.

The application is designed to be responsive across desktop and mobile devices, employing Tailwind CSS for utility-first styling. The platform demonstrates a production-ready architecture suitable for deployment on cloud services like Vercel (frontend), Render/Railway (backend), and MongoDB Atlas (database).

Keywords: Roadside Assistance, Mechanic Booking, MERN Stack, Geospatial Queries, Real-time Tracking, JWT Authentication, Next.js, MongoDB, REST API

Chapter 2: Introduction

2.1 Overview

Vehicle breakdowns are unpredictable events that can leave drivers stranded, stressed, and vulnerable, especially on highways or in unfamiliar areas. Traditional methods of finding roadside assistance typically involve manual phone calls, unreliable word-of-mouth referrals, and uncertain wait times. These pain points create a significant market opportunity for a technology-driven solution.

The Roadside Vehicle Assistance & Mechanic Booking Platform (Roadside AAA) is designed to bridge this gap by providing a centralized, digital platform that connects vehicle owners with verified mechanics and towing service providers in real time. The platform leverages modern web technologies and geospatial capabilities to deliver fast, transparent, and reliable roadside assistance.

2.2 Problem Statement

Vehicle breakdowns cause significant stress and delay due to the following challenges:

- Difficulty finding reliable and trustworthy mechanics in the vicinity
- Unclear and non-transparent pricing for roadside services
- Delayed support due to manual communication channels
- Lack of real-time tracking of service provider arrival
- No centralized platform for service history and accountability
- Over-dependence on manual phone calls and referrals
- No mechanism for rating and feedback on services received

2.3 Objectives

2.3.1 Primary Objectives

- Provide fast and efficient roadside assistance during vehicle breakdowns
- Connect vehicle owners with verified nearby mechanics and towing services
- Offer transparent pricing and detailed service information
- Reduce vehicle downtime and user stress through technology
- Enable real-time tracking of service request status

2.3.2 Secondary Objectives

- Improve trust in local automotive services through ratings and reviews
- Enable comprehensive service history and accountability
- Support scalable expansion across cities and regions
- Provide mechanics with a digital platform for job management and earnings tracking
- Give administrators tools for platform monitoring and quality control

2.4 Scope of the Project

2.4.1 In Scope (Phase 1)

- Responsive web platform (desktop and mobile)
- Real-time mechanic discovery and booking via geospatial queries
- Six service types: breakdown repair, towing, battery jump-start, flat tire repair, fuel delivery, lockout assistance
- Service tracking with status updates (pending → accepted → en route → arrived → in progress → completed)
- User, mechanic, and admin dashboards with role-based access
- JWT-based authentication and authorization
- Rating and feedback system
- Admin partner verification workflow

2.4.2 Out of Scope (Phase 1)

- Native mobile applications (iOS/Android)
- Vehicle insurance claim processing
- Spare parts inventory management
- Online payment gateway integration
- Push notifications (WebSocket/SSE)

2.5 Project Significance

This project is significant as it demonstrates the practical application of full-stack web development skills including frontend development with Next.js and React, backend API design with Express.js, database design with MongoDB, authentication and authorization, geospatial computing, and modern UI/UX design. It addresses a real-world problem that impacts millions of vehicle owners and creates value for both service consumers and providers.

Chapter 3: Literature Survey

3.1 Existing Systems

Several existing platforms provide roadside assistance services. A comparative analysis of these systems provides context for the design decisions made in this project.

3.1.1 AAA (American Automobile Association)

AAA is a well-established roadside assistance provider in the United States offering membership-based services including towing, battery service, flat tire change, fuel delivery, and locksmith service. While comprehensive, AAA operates on a membership model with annual fees and is geographically limited to the US and Canada. The platform lacks real-time transparent pricing and relies heavily on dispatched service providers.

3.1.2 Mechanic On-Demand Apps

Various mobile-first platforms like YourMechanic and Wrench have emerged to provide on-demand mechanic services. These platforms focus on scheduled repairs rather than emergency roadside assistance and typically require advance booking. They demonstrate the viability of a marketplace model for automotive services.

3.1.3 Ride-Hailing Model Inspiration

Platforms like Uber and Ola have pioneered the real-time geolocation-based service matching model. The key concepts of nearby provider discovery, ETA estimation, real-time tracking, and post-service rating systems have been adapted for the roadside assistance domain in this project.

3.2 Technology Review

3.2.1 Next.js and React Ecosystem

Next.js 14, built on top of React, provides a robust framework for building modern web applications with features like the App Router for file-system based routing, server-side rendering (SSR), static site generation (SSG), and API routes. The App Router architecture enables colocation of layouts, loading states, and error boundaries, making it ideal for building complex dashboard interfaces.

3.2.2 Node.js and Express.js

Node.js provides a non-blocking, event-driven JavaScript runtime ideal for building scalable network applications. Express.js, a minimal and flexible web framework for Node.js, enables rapid API development with middleware support, routing, and error handling. The combination is well-suited for building RESTful APIs with JSON data exchange.

3.2.3 MongoDB and Geospatial Capabilities

MongoDB is a document-oriented NoSQL database that provides flexibility in schema design and native support for GeoJSON data types and 2dsphere indexes. These geospatial capabilities are critical for the nearby mechanic discovery feature, enabling efficient location-based queries. Mongoose ODM provides schema validation and middleware hooks for data integrity.

3.2.4 JWT Authentication

JSON Web Tokens (JWT) provide a stateless, self-contained method for securely transmitting information between parties as a JSON object. JWTs are digitally signed and can carry claims (user ID, role) that enable role-based access control without maintaining server-side sessions, making the architecture horizontally scalable.

3.3 Research Gap

The literature survey reveals that while individual components of roadside assistance platforms exist, there is a gap in providing an integrated, open-source, full-stack solution that combines real-time geospatial mechanic discovery, transparent pricing, service lifecycle tracking, multi-role dashboards, and admin oversight within a single platform. This project aims to fill that gap.

Chapter 4: System Analysis

4.1 Requirement Analysis

4.1.1 Functional Requirements — Vehicle Owner

S.No.	Feature	Description	Status
1	Registration & Login	Users can register with name, email, phone, and password. Login via email/password returns a JWT token.	Implemented
2	Vehicle Management	Add, view, edit, and delete vehicles with make, model, year, license plate, color, and fuel type.	Implemented
3	Request Assistance	Create service requests by selecting vehicle, service type, providing location, and optionally choosing a mechanic.	Implemented
4	Nearby Mechanic Discovery	View verified mechanics within 50 km radius with distance, rating, ETA, and pricing information.	Implemented
5	Real-time Tracking	Track service request status updates via polling (5–15 second intervals).	Implemented
6	Service History	View past and active service requests with full details and status history.	Implemented
7	Rating & Feedback	Rate completed services (1–5 stars) and provide written feedback.	Implemented

4.1.2 Functional Requirements — Mechanic / Towing Partner

S.No.	Feature	Description	Status
1	Partner Registration	Register as mechanic with business name and	Implemented

		license number. Requires admin verification.	
2	Profile Management	Update business name, bio, experience, services offered, and pricing for each service type.	Implemented
3	View Pending Requests	Browse all pending service requests from users that are awaiting mechanic acceptance.	Implemented
4	Accept Requests	Accept pending requests to begin service. Calculates price based on mechanic pricing.	Implemented
5	Status Updates	Update job status through lifecycle: en route → arrived → in progress → completed.	Implemented
6	Earnings Dashboard	Track total earnings, total completed jobs, and view recent job history.	Implemented
7	Location Update	Update current location coordinates for accurate distance calculations.	Implemented

4.1.3 Functional Requirements — Admin

S.No.	Feature	Description	Status
1	KPI Dashboard	View total users, mechanics, pending applications, active/completed requests, avg rating, acceptance rate.	Implemented
2	Partner Verification	Review, verify, or reject mechanic/towing partner applications.	Implemented
3	User Management	View all registered vehicle owners.	Implemented
4	Request Monitoring	View all service requests across the platform with full details.	Implemented
5	Dispute Resolution	Resolve or cancel disputed service requests with admin notes.	Implemented

4.1.4 Non-Functional Requirements

Requirement	Implementation Details
Performance	Polling-based updates at 5–15 second intervals; geospatial queries with 2dsphere indexes; Haversine formula for distance calculation
Security	JWT-based authentication with 7-day token expiry; bcrypt password hashing with 12 salt rounds; role-based authorization middleware
Usability	Emergency-friendly UI with large call-to-action buttons; mobile-responsive design; intuitive dashboard navigation
Scalability	Stateless REST API architecture; MongoDB compound and geospatial indexes; modular codebase structure
Reliability	Error handling middleware; input validation; status history audit trail for all service requests

4.2 Feasibility Study

4.2.1 Technical Feasibility

The project is technically feasible as it uses well-established, production-ready technologies. Next.js 14 and React provide a mature frontend framework, Express.js is battle-tested for API development,

and MongoDB Atlas offers managed cloud database services. All libraries used are actively maintained with large communities and comprehensive documentation.

4.2.2 Operational Feasibility

The platform is designed to be intuitive for all user roles. Vehicle owners can request assistance with minimal steps, mechanics receive clear job information, and administrators have comprehensive dashboards. The responsive design ensures accessibility across devices.

4.2.3 Economic Feasibility

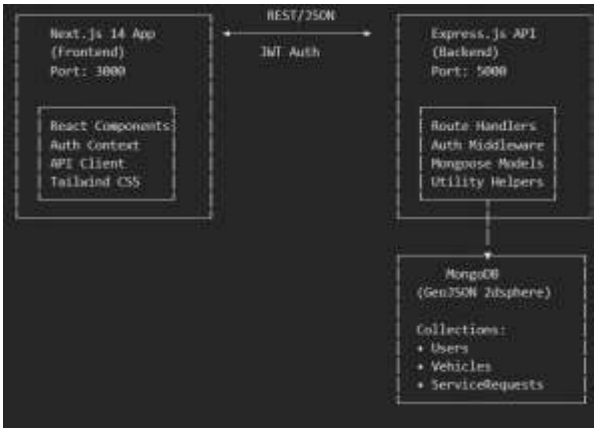
The project uses entirely open-source technologies, eliminating licensing costs. Deployment costs are minimal with free tiers available on Vercel (frontend), Render (backend), and MongoDB Atlas (database). The platform can generate revenue through commission-based models on service bookings.

Chapter 5: System Design

5.1 System Architecture

The application follows a client-server architecture with a clear separation between the frontend (presentation layer), backend (business logic layer), and database (data persistence layer). Communication between the frontend and backend occurs via RESTful API calls using JSON as the data interchange format.

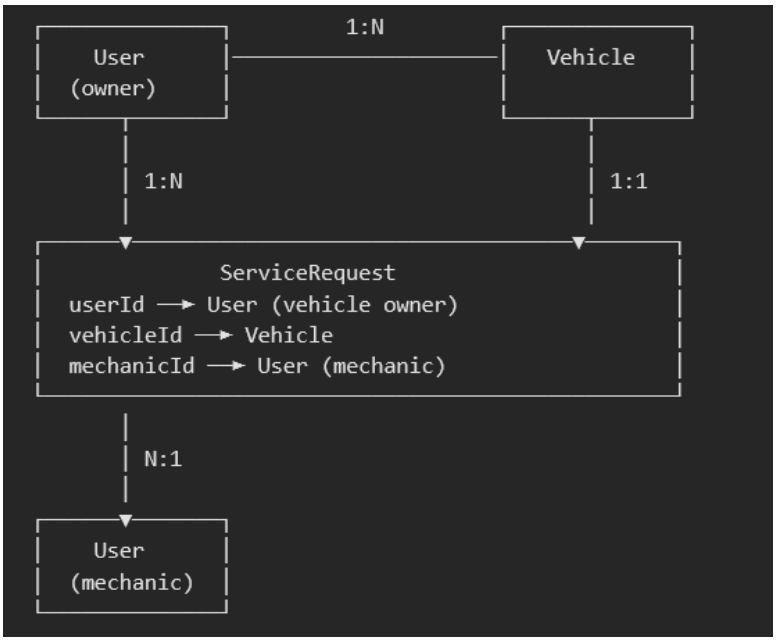
Figure 1: System Architecture Diagram



5.2 Database Design

5.2.1 Entity-Relationship Diagram

Figure 2: Entity-Relationship (ER) Diagram



5.2.2 User Collection Schema

Field	Type	Constraints	Description
name	String	Required, Trimmed	Full name of the user
email	String	Required, Unique, Lowercase	Email address (login identifier)
phone	String	Required	Contact phone number
password	String	Required, Min 6 chars	Hashed password (bcrypt, 12 rounds)
role	String	Enum: user/mechanic/admin	User role for access control
isVerified	Boolean	Default: false	Verification status (admin-approved for mechanics)
profile.businessName	String	Optional	Mechanic business name
profile.licenseNumber	String	Optional	Mechanic license number
profile.experience	Number	Optional	Years of experience
profile.services	[String]	Optional	Services offered by mechanic
profile.pricing.*	Number	Defaults set	Per-service pricing (6 service types)
profile.rating	Number	Default: 0	Average rating from users
profile.totalRatings	Number	Default: 0	Count of ratings received
profile.totalJobs	Number	Default: 0	Total completed jobs
profile.earnings	Number	Default: 0	Total earnings accumulated
location.type	String	Enum: Point	GeoJSON type
location.coordinates	[Number]	Default: [77.59, 12.97]	Longitude, Latitude pair
location.address	String	Optional	Human-readable address
isAvailable	Boolean	Default: true	Mechanic availability status

5.2.3 Vehicle Collection Schema

Field	Type	Constraints	Description
userId	ObjectId	Required, Ref: User	Owner of the vehicle
make	String	Required	Vehicle manufacturer (e.g., Maruti Suzuki)
model	String	Required	Vehicle model (e.g., Swift)
year	Number	Required	Manufacturing year
licensePlate	String	Required	Registration number
color	String	Optional	Vehicle color
fuelType	String	Enum: petrol/diesel/electric/hybrid/cng	Fuel type

5.2.4 ServiceRequest Collection Schema

Field	Type	Constraints	Description
userId	ObjectId	Required, Ref: User	User who created the request
vehicleId	ObjectId	Required, Ref: Vehicle	Vehicle needing service
mechanicId	ObjectId	Optional, Ref: User	Assigned mechanic

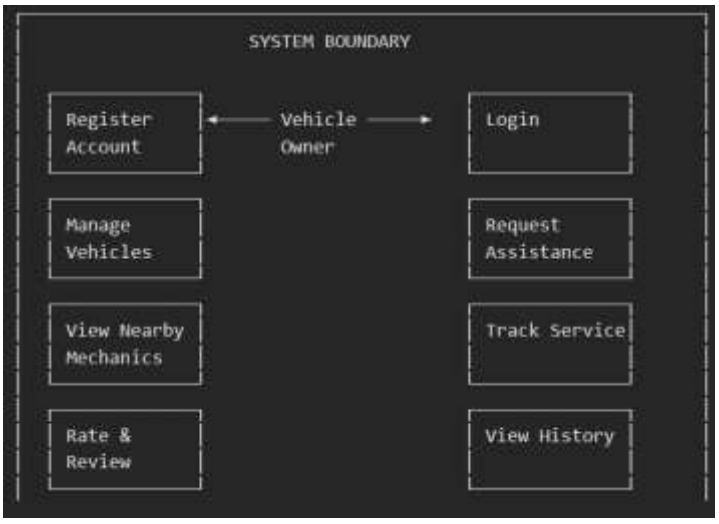
serviceType	String	Required, Enum (6 types)	Type of service requested
description	String	Default: empty	Additional description of the issue
location.coordinates	[Number]	Required	Breakdown location (GeoJSON)
location.address	String	Required	Breakdown address
status	String	Enum (7 states)	Current request status
estimatedPrice	Number	Required	Estimated service cost
finalPrice	Number	Optional	Final price after completion
estimatedArrivalMinutes	Number	Optional	ETA in minutes
rating	Number	Min: 1, Max: 5	User rating after service
feedback	String	Optional	User feedback text
statusHistory	[Object]	Array	Audit trail of status changes

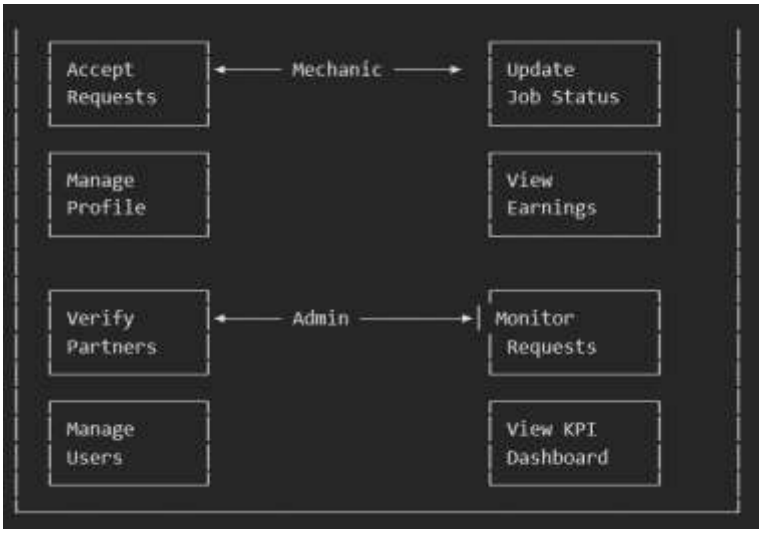
5.2.5 Database Indexes

Collection	Index	Type	Purpose
User	location	2dsphere	Geospatial queries for nearby mechanics
User	role + isVerified	Compound	Efficient mechanic filtering
ServiceRequest	location	2dsphere	Geospatial queries on breakdown locations
ServiceRequest	status + createdAt	Compound	Admin dashboard queries

5.3 Use Case Diagram

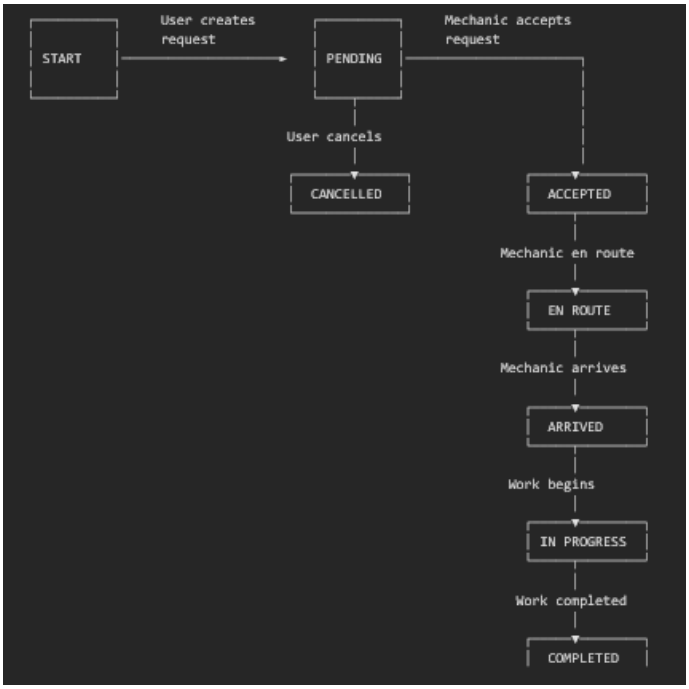
Figure 3: Use Case Diagram





5.4 Service Request Status Flow

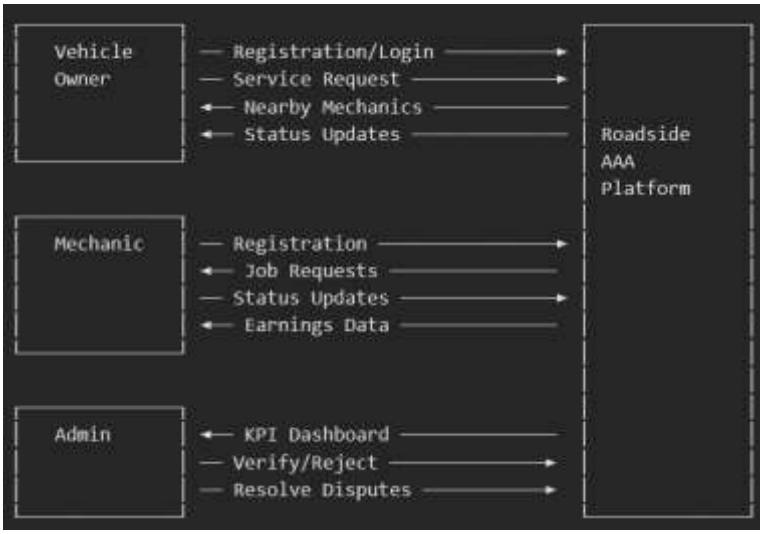
Figure 4: Service Request Status Flow Diagram



5.5 Data Flow Diagrams

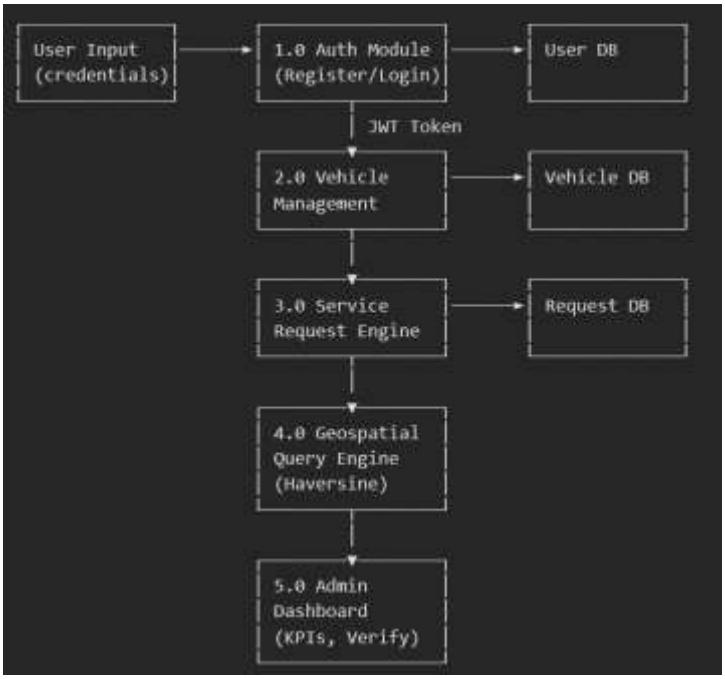
5.5.1 Level 0 (Context Diagram)

Figure 5: Data Flow Diagram — Level 0



5.5.2 Level 1 (Detailed DFD)

Figure 6: Data Flow Diagram — Level 1



5.6 API Design

The backend exposes a RESTful API with the following endpoints:

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register user or mechanic account
POST	/api/auth/login	Public	Login and receive JWT token
GET	/api/auth/me	Protected	Get current authenticated user

PUT	/api/auth/me	Protected	Update user profile and location
GET	/api/vehicles	User	List vehicles for current user
POST	/api/vehicles	User	Add a new vehicle
PUT	/api/vehicles/:id	User	Update vehicle details
DELETE	/api/vehicles/:id	User	Delete a vehicle
GET	/api/mechanics/nearby	User	Find nearby verified mechanics
GET	/api/mechanics/requests	Mechanic	View pending service requests
GET	/api/mechanics/jobs	Mechanic	View mechanic job history
PUT	/api/mechanics/profile	Mechanic	Update mechanic profile and pricing
PUT	/api/mechanics/location	Mechanic	Update mechanic location
GET	/api/mechanics/earnings	Mechanic	Get earnings summary
GET	/api/requests/types	Protected	Get available service types
POST	/api/requests	User	Create a new service request
GET	/api/requests	Protected	List requests (filtered by role)
GET	/api/requests/:id	Protected	Get request details
PUT	/api/requests/:id/accept	Mechanic	Accept a pending request
PUT	/api/requests/:id/status	Protected	Update request status
PUT	/api/requests/:id/rate	User	Rate a completed service
GET	/api/admin/dashboard	Admin	Get KPI dashboard data
GET	/api/admin/mechanics/pending	Admin	List pending mechanic applications
PUT	/api/admin/mechanics/:id/verify	Admin	Approve a mechanic
PUT	/api/admin/mechanics/:id/reject	Admin	Reject a mechanic application
GET	/api/admin/users	Admin	List all registered users
GET	/api/admin/requests	Admin	List all service requests
PUT	/api/admin/requests/:id/resolve	Admin	Resolve/cancel a request
GET	/api/health	Public	Health check endpoint

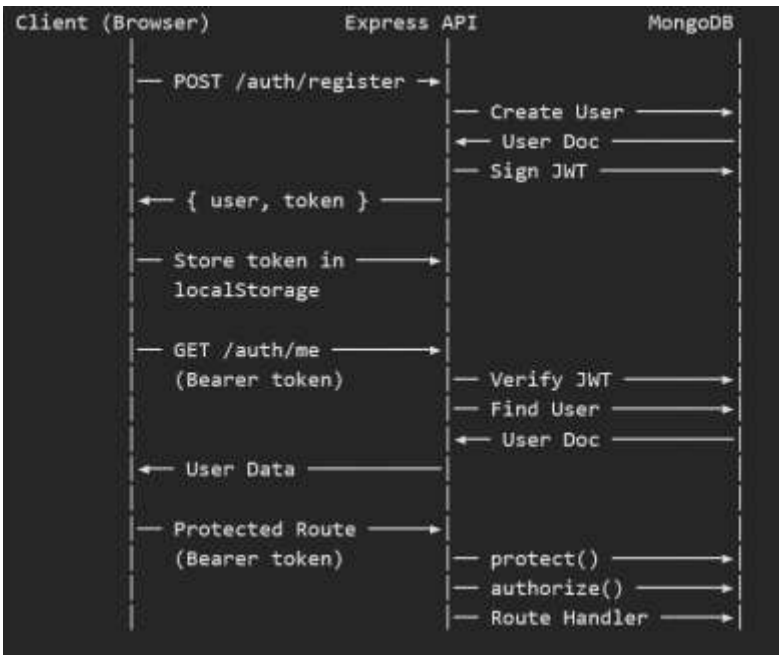
5.7 Frontend Component Architecture

Figure 7: Frontend Component Hierarchy

```
app/
├── layout.tsx (Root Layout + AuthProvider)
├── page.tsx (Landing Page)
│   ├── Navbar
│   ├── Hero
│   ├── Stats
│   ├── Features
│   ├── Services
│   ├── HowItWorks
│   ├── LiveDemo
│   ├── Testimonials
│   ├── TrustSection
│   ├── FAQ
│   ├── CTA
│   └── Footer
├── login/page.tsx (Login Form)
├── register/page.tsx (Registration Form)
└── dashboard/
    ├── page.tsx (Role-based Redirect)
    ├── user/
    │   ├── page.tsx (User Dashboard + Request Flow)
    │   ├── vehicles/page.tsx (Vehicle Management)
    │   ├── request/page.tsx (New Service Request)
    │   ├── track/page.tsx (Live Service Tracking)
    │   ├── history/page.tsx (Service History)
    │   ├── profile/page.tsx (User Profile)
    │   └── settings/page.tsx (User Settings)
    ├── mechanic/
    │   ├── page.tsx (Mechanic Dashboard)
    │   ├── requests/page.tsx (Pending Requests)
    │   ├── jobs/page.tsx (Job History)
    │   ├── earnings/page.tsx (Earnings Dashboard)
    │   └── profile/page.tsx (Mechanic Profile)
    └── admin/
        ├── page.tsx (Admin KPI Dashboard)
        ├── verify/page.tsx (Partner Verification)
        ├── users/page.tsx (User Management)
        └── requests/page.tsx (Request Monitoring)
```

5.8 Authentication Flow

Figure 8: Authentication Flow Sequence Diagram



5.9 Frontend Route Structure

Path	Role	Page Component	Purpose
/	Public	Landing Page	Marketing page with features, services, testimonials
/login	Public	Login Form	Email/password authentication
/register	Public	Registration Form	User or mechanic registration
/dashboard	Protected	Role Redirect	Redirects to role-specific dashboard
/dashboard/user	User	User Dashboard	Overview with quick request flow
/dashboard/user/vehicles	User	Vehicle Management	CRUD operations for vehicles
/dashboard/user/request	User	New Request	Create service request
/dashboard/user/track	User	Live Tracking	Real-time status tracking
/dashboard/user/history	User	Service History	Past request records
/dashboard/user/profile	User	User Profile	Profile management
/dashboard/user/settings	User	Settings	Account settings
/dashboard/mechanic	Mechanic	Mechanic Dashboard	Job overview and stats
/dashboard/mechanic/requests	Mechanic	Pending Requests	Browse and accept jobs
/dashboard/mechanic/jobs	Mechanic	Job History	Completed job records
/dashboard/mechanic/earnings	Mechanic	Earnings	Revenue tracking
/dashboard/mechanic/profile	Mechanic	Mechanic Profile	Business profile management
/dashboard/admin	Admin	Admin Dashboard	KPI overview and monitoring
/dashboard/admin/verify	Admin	Partner Verification	Approve/reject mechanics
/dashboard/admin/users	Admin	User Management	View registered users
/dashboard/admin/requests	Admin	All Requests	Platform-wide request monitoring

Chapter 6: Implementation

6.1 Technology Stack

6.1.1 Frontend Technologies

Technology	Version	Purpose
Next.js	14.2.15	React framework with App Router, SSR, and file-based routing
React	18.3.1	UI component library for building interactive interfaces
TypeScript	5.6.3	Type-safe JavaScript superset for better DX and error prevention
Tailwind CSS	3.4.14	Utility-first CSS framework for responsive design
Lucide React	0.454.0	Modern icon library for consistent UI iconography
PostCSS	8.4.47	CSS transformation tool for Tailwind processing
Autoprefixer	10.4.20	CSS vendor prefix automation

6.1.2 Backend Technologies

Technology	Version	Purpose
Node.js	18+	JavaScript runtime for server-side execution
Express.js	4.21.0	Minimal web framework for REST API development
Mongoose	8.7.0	MongoDB ODM for schema definition and data validation
jsonwebtoken	9.0.2	JWT generation and verification for authentication
bcryptjs	2.4.3	Password hashing library (12 salt rounds)
cors	2.8.5	Cross-Origin Resource Sharing middleware
dotenv	16.4.5	Environment variable management
nodemon	3.1.7	Development auto-reload server (dev dependency)

6.2 Project Structure

Project Directory Structure

```
roadside-vehicle-assistance/
    ├── backend/
    │   ├── config/
    │   │   └── db.js                  # MongoDB connection configuration
    │   ├── middleware/
    │   │   └── auth.js                # JWT protect() and authorize() middleware
    │   ├── models/
    │   │   ├── User.js                # User schema with embedded mechanic profile
    │   │   ├── Vehicle.js             # Vehicle schema linked to user
    │   │   └── ServiceRequest.js      # Service request with status lifecycle
    │   ├── routes/
    │   │   ├── auth.js                # Registration, login, profile endpoints
    │   │   ├── vehicles.js            # CRUD operations for vehicles
    │   │   ├── mechanics.js           # Nearby search, jobs, earnings, profile
    │   │   ├── serviceRequests.js     # Request CRUD, accept, status, rate
    │   │   └── admin.js               # Dashboard, verification, monitoring
    │   ├── utils/
    │   │   ├── helpers.js             # Haversine distance, ETA, pricing helpers
    │   │   └── seed.js                # Database seed script with demo data
    │   ├── server.js                  # Express app entry point
    │   ├── package.json               # Backend dependencies
    │   └── .env                       # Environment variables
    │
    ├── frontend/
    │   ├── app/
    │   │   ├── layout.tsx             # Root layout with AuthProvider
    │   │   ├── page.tsx               # Landing page
    │   │   ├── globals.css            # Global styles and Tailwind imports
    │   │   ├── login/page.tsx         # Login page
    │   │   ├── register/page.tsx      # Registration page
    │   │   └── dashboard/
    │   │       ├── page.tsx           # Dashboard role-redirect
    │   │       ├── user/              # User dashboard pages (7 pages)
    │   │       ├── mechanic/          # Mechanic dashboard pages (5 pages)
    │   │       └── admin/             # Admin dashboard pages (4 pages)
    │   ├── components/
    │   │   ├── Navbar.tsx             # Navigation bar with auth state
```

```
│   │   ├── Hero.tsx              # Landing page hero section
│   │   ├── Features.tsx          # Feature showcase
│   │   ├── Services.tsx          # Service types display
│   │   ├── HowItWorks.tsx        # Step-by-step guide
│   │   ├── Stats.tsx             # Platform statistics
│   │   ├── LiveDemo.tsx          # Interactive demo component
│   │   ├── Testimonials.tsx      # User testimonials
│   │   ├── TrustSection.tsx      # Trust indicators
│   │   ├── FAQ.tsx               # Frequently asked questions
│   │   ├── CTA.tsx               # Call to action
│   │   ├── Footer.tsx            # Site footer
│   │   ├── DashboardLayout.tsx   # Shared dashboard shell (sidebar + nav)
│   │   └── ui.tsx                # Reusable UI primitives
│   ├── lib/
│   │   ├── api.ts                # API client with typed endpoints
│   │   └── auth-context.tsx      # React Context for authentication
│   ├── package.json              # Frontend dependencies
│   ├── tailwind.config.js        # Tailwind CSS configuration
│   ├── tsconfig.json             # TypeScript configuration
│   └── next.config.js            # Next.js configuration
│
├── docs/
│   ├── PRD.md                    # Product Requirements Document
│   └── TECHNICAL.md              # Technical Documentation
│
├── server.js                     # Root wrapper for Render deployment
├── package.json                  # Root package.json
└── README.md                     # Project documentation
```

6.3 Key Implementation Details

6.3.1 Haversine Distance Calculation

The platform uses the Haversine formula to calculate the great-circle distance between the user's breakdown location and each mechanic's current location. This formula accounts for the curvature of the Earth and provides accurate results for short to medium distances.

JavaScript — helpers.js

```javascript
const R = 6371; // Earth radius in kilometers

function haversineDistance(lat1, lon1, lat2, lon2) {
  const dLat = ((lat2 - lat1) * Math.PI) / 180;
  const dLon = ((lon2 - lon1) * Math.PI) / 180;
  const a =
    Math.sin(dLat / 2) ** 2 +
    Math.cos((lat1 * Math.PI) / 180) *
      Math.cos((lat2 * Math.PI) / 180) *
      Math.sin(dLon / 2) ** 2;
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
}
```

6.3.2 ETA Estimation Algorithm

The estimated time of arrival is calculated by assuming an average travel speed of 30 km/h in urban conditions, with a minimum ETA of 5 minutes.

JavaScript — helpers.js

```
function estimateArrivalMinutes(distanceKm) {
  const avgSpeedKmh = 30;
  return Math.max(5, Math.round((distanceKm / avgSpeedKmh) * 60));
}
```

6.3.3 JWT Authentication Middleware

The authentication middleware extracts the JWT from the Authorization header, verifies it against the server secret, and attaches the authenticated user to the request object. The authorize function performs role-based access control.

JavaScript — middleware/auth.js

```
const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization?.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }
  if (!token) {
    return res.status(401).json({ message: 'Not authorized, no token' });
  }
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id);
    if (!req.user) {
      return res.status(401).json({ message: 'User not found' });
    }
    next();
  } catch {
    return res.status(401).json({ message: 'Not authorized, token invalid' });
  }
};

const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: 'Access denied for this role' });
  }
  next();
};
```

6.3.4 Service Request Lifecycle

The service request lifecycle is managed through a state machine pattern. Each status transition is recorded in the statusHistory array, creating a complete audit trail. Upon completion, the mechanic's earnings and job counters are automatically updated.

JavaScript — models/ServiceRequest.js

```
const STATUS_FLOW = [
  'pending',      // User created the request
  'accepted',     // Mechanic accepted the job
  'en_route',     // Mechanic is traveling to location
  'arrived',      // Mechanic arrived at breakdown site
  'in_progress',  // Repair/service work in progress
  'completed',    // Service successfully completed
  'cancelled',    // Request cancelled by user/admin
];
```

6.3.5 Nearby Mechanic Discovery

When a user searches for nearby mechanics, the system queries all verified and available mechanics, calculates the Haversine distance from the user's location, filters mechanics within a 50 km radius, and sorts results by proximity. For each mechanic, the ETA and service pricing are computed dynamically.

6.3.6 Rating System

After a service is marked as completed, the user can rate the mechanic on a scale of 1 to 5 stars with optional written feedback. The mechanic's average rating is recalculated using a weighted running average formula to maintain accuracy as ratings accumulate.

JavaScript — Rating Calculation

```
// Running average calculation
const newTotal = currentTotalRatings + 1;
const newRating = (currentRating * currentTotalRatings + newRating) / newTotal;
// Rounded to 1 decimal place
mechanic.profile.rating = Math.round(newRating * 10) / 10;
```

6.4 Environment Configuration

6.4.1 Backend Environment Variables (.env)

Environment — Backend

```
PORT=5000
MONGODB_URI=mongodb://localhost:27017/roadside_assistance
JWT_SECRET=<strong-secret-key>
NODE_ENV=development
```

6.4.2 Frontend Environment Variables (.env.local)

Environment — Frontend

```
NEXT_PUBLIC_API_URL=http://localhost:5000/api
```

Chapter 7: Testing

7.1 Testing Strategy

The application was tested using a combination of manual testing, API testing (using tools like Postman or curl), and UI testing. Test cases were designed to cover all functional requirements across all three user roles.

7.2 Test Cases — Authentication Module

TC ID	Test Case	Input	Expected Output	Result
TC-01	User Registration (Valid)	Name, email, phone, password	201 Created with JWT token	Pass
TC-02	User Registration (Duplicate Email)	Existing email address	400 Bad Request — Email already registered	Pass
TC-03	User Registration (Missing Fields)	Empty name field	400 Bad Request — Please provide all required fields	Pass
TC-04	Login (Valid Credentials)	Correct email & password	200 OK with JWT token and user data	Pass
TC-05	Login (Invalid Password)	Wrong password	401 Unauthorized — Invalid email or password	Pass
TC-06	Login (Non-existent Email)	Unregistered email	401 Unauthorized — Invalid email or password	Pass
TC-07	Access Protected Route (Valid Token)	Valid JWT in header	200 OK with user data	Pass
TC-08	Access Protected Route (No Token)	No Authorization header	401 Unauthorized — Not authorized, no token	Pass
TC-09	Access Protected Route (Expired Token)	Expired JWT token	401 Unauthorized — Not authorized, token invalid	Pass
TC-10	Mechanic Registration	Role=mechanic, businessName, license	201 Created with isVerified=false	Pass

7.3 Test Cases — Service Request Module

TC ID	Test Case	Input	Expected Output	Result
TC-11	Create Service Request (Valid)	vehicleId, serviceType, location	201 Created with pending status	Pass
TC-12	Create Request (Missing Vehicle)	Invalid vehicleId	404 Vehicle not found	Pass
TC-13	Create Request (Missing Location)	No coordinates	400 Missing required fields	Pass
TC-14	Find Nearby Mechanics	User coordinates (lng, lat)	200 OK with sorted mechanic list	Pass
TC-15	Accept Request	Pending request ID	200 OK with	Pass

	(Verified Mechanic)		status=accepted	
TC-16	Accept Request (Unverified Mechanic)	Pending request, unverified mechanic	403 Account not verified	Pass
TC-17	Update Status (en_route)	request ID, status=en_route	200 OK with updated status	Pass
TC-18	Complete Service	status=completed, finalPrice	200 OK, mechanic earnings updated	Pass
TC-19	Rate Completed Service	rating=5, feedback text	200 OK, mechanic rating recalculated	Pass
TC-20	Cancel Request (By User)	status=cancelled	200 OK with cancelled status	Pass

7.4 Test Cases — Admin Module

TC ID	Test Case	Input	Expected Output	Result
TC-21	Admin Dashboard KPIs	Admin JWT token	200 OK with stats object	Pass
TC-22	View Pending Mechanics	Admin token	200 OK with pending mechanic list	Pass
TC-23	Verify Mechanic	mechanic ID	200 OK with isVerified=true	Pass
TC-24	Reject Mechanic	mechanic ID	200 OK, mechanic deleted	Pass
TC-25	View All Users	Admin token	200 OK with user list (no passwords)	Pass
TC-26	Resolve Disputed Request	request ID, status, note	200 OK with updated request	Pass
TC-27	Non-Admin Access Dashboard	User JWT token	403 Access denied for this role	Pass

7.5 Demo Test Accounts

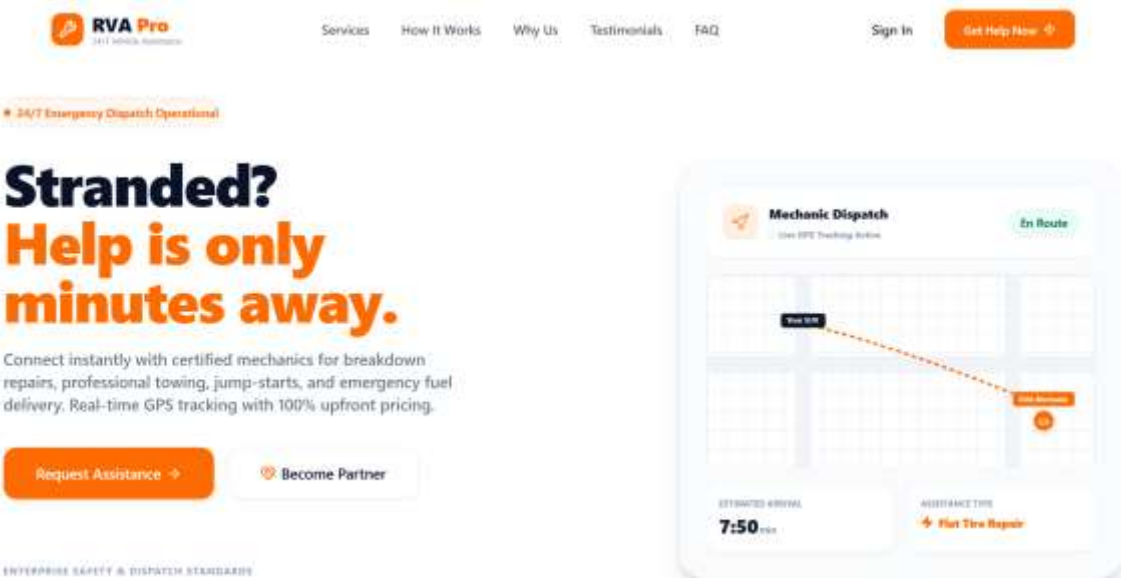
Role	Email	Password	Notes
Admin	admin@roadside.com	admin123	Full platform access
User	user@roadside.com	user123	Vehicle owner with pre-registered vehicle
Mechanic (Verified)	rajesh@roadside.com	mech123	Quick Fix Auto — 8 years experience
Mechanic (Verified)	suresh@roadside.com	mech123	Highway Tow Services — 12 years experience
Mechanic (Pending)	amit@roadside.com	mech123	City Auto Care — Awaiting verification

Chapter 8: Screenshots & Output

This chapter presents the key screens and outputs of the Roadside Vehicle Assistance & Mechanic Booking Platform. Due to the document format, representative descriptions of each screen are provided below. Actual screenshots can be captured by running the application locally.

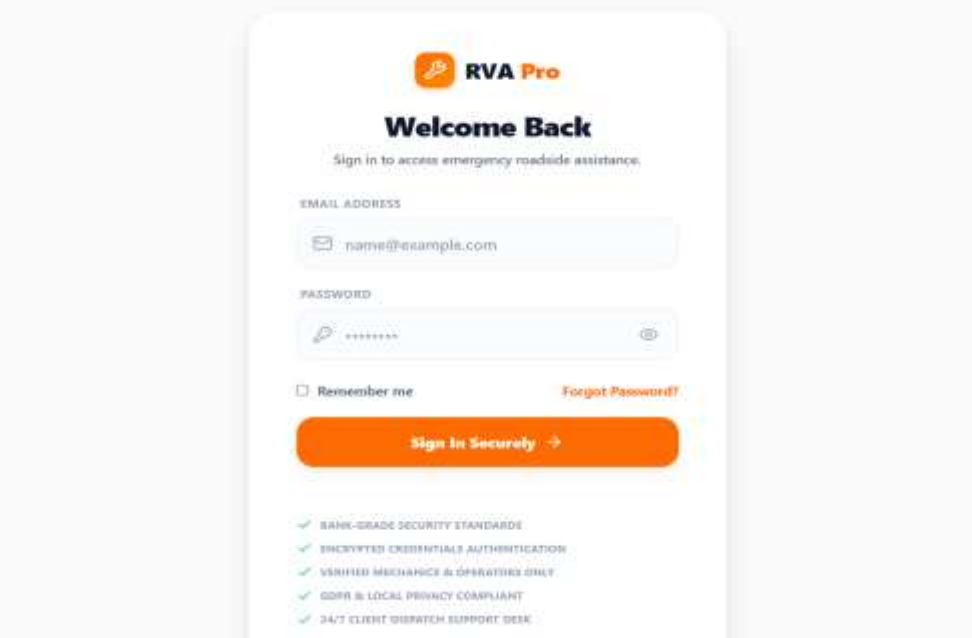
8.1 Landing Page (Home)

The landing page features a modern, gradient-themed hero section with a prominent 'Get Help Now' call-to-action button. It includes platform statistics (total users, response time, satisfaction rating), a feature showcase section, service type cards (breakdown repair, towing, battery jump-start, etc.), a 'How It Works' step-by-step guide, an interactive live demo, user testimonials, trust indicators, a FAQ accordion, and a comprehensive footer with navigation links.



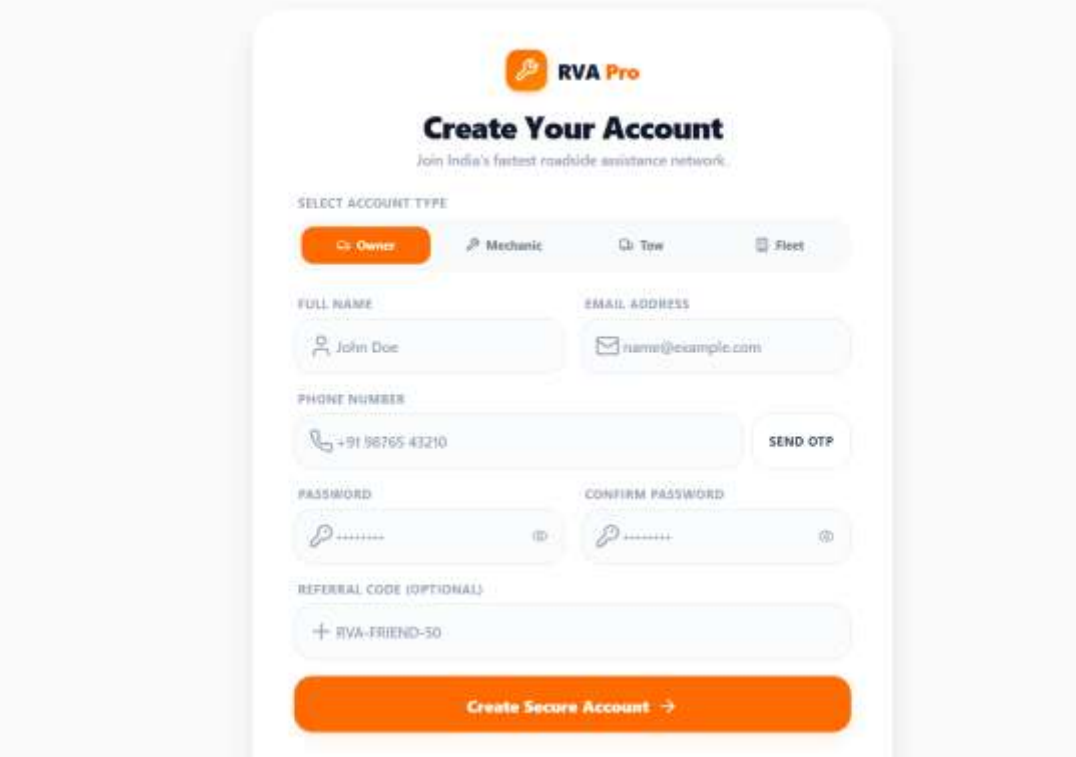
8.2 Login Page

A clean, centered login form with email and password fields, form validation, and links to the registration page. The design uses a card-based layout with subtle shadows and the platform branding.



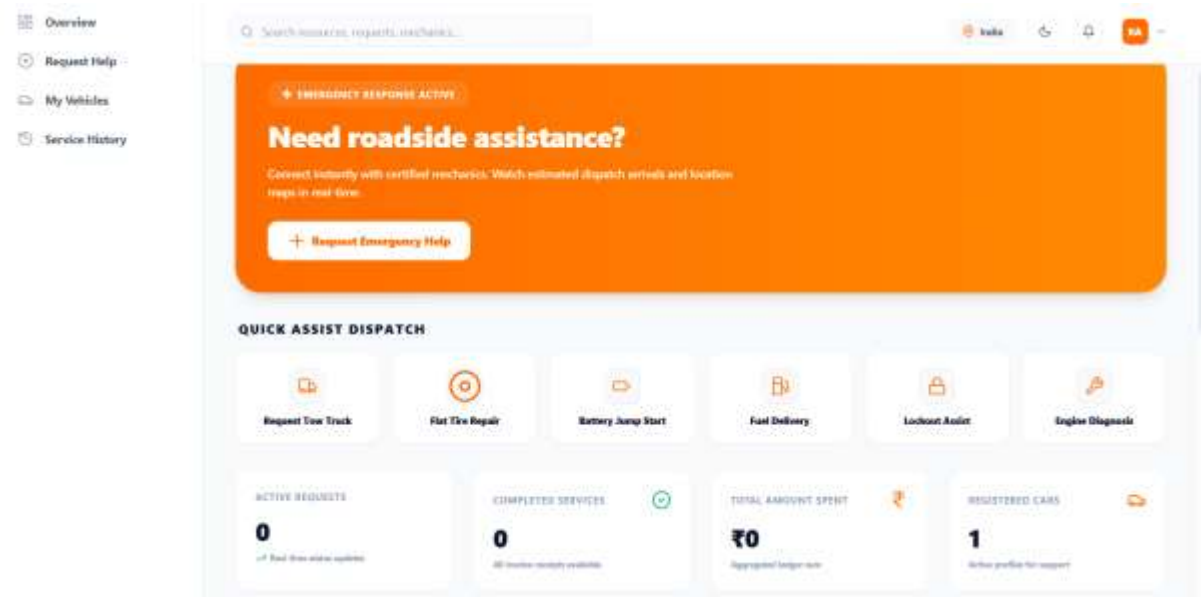
8.3 Registration Page

A registration form supporting both vehicle owner and mechanic registration. When 'mechanic' role is selected, additional fields for business name and license number appear dynamically. The form includes client-side validation for all required fields.



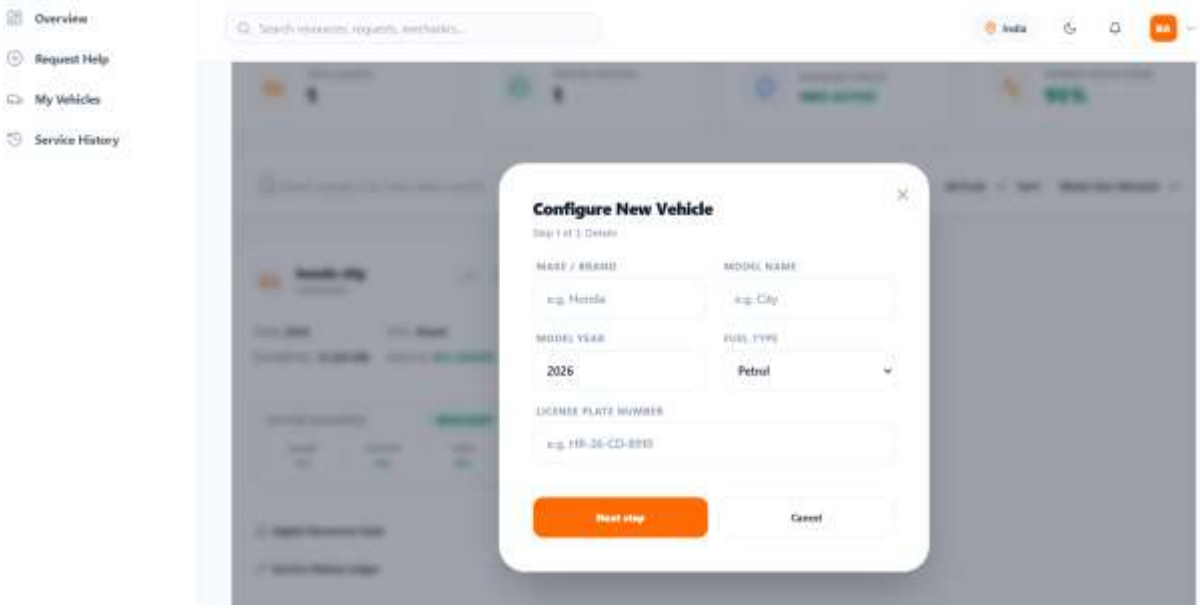
8.4 User Dashboard

The user dashboard provides an overview of the user's activity with quick access cards for requesting assistance, managing vehicles, viewing service history, and tracking active requests. It includes a multi-step service request flow directly from the dashboard: select vehicle → choose service type → share location → pick a mechanic → confirm and submit.



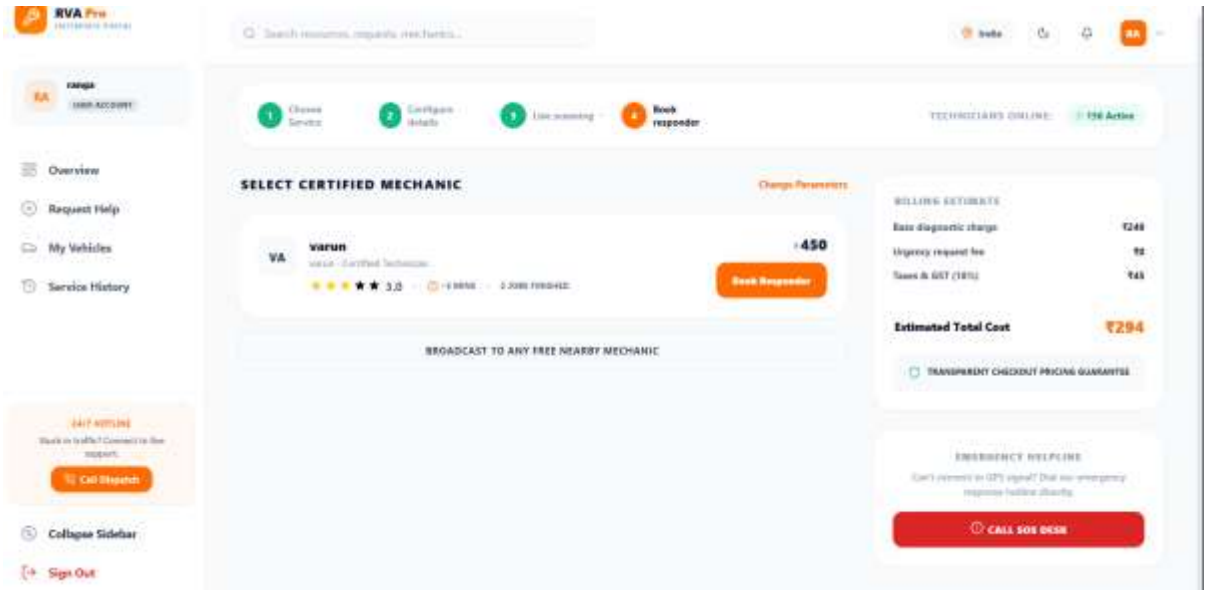
8.5 Vehicle Management

A paginated list of the user's registered vehicles displayed as cards, each showing make, model, year, license plate, color, and fuel type. Includes buttons for adding new vehicles and editing/deleting existing ones. The add/edit form validates all required fields.



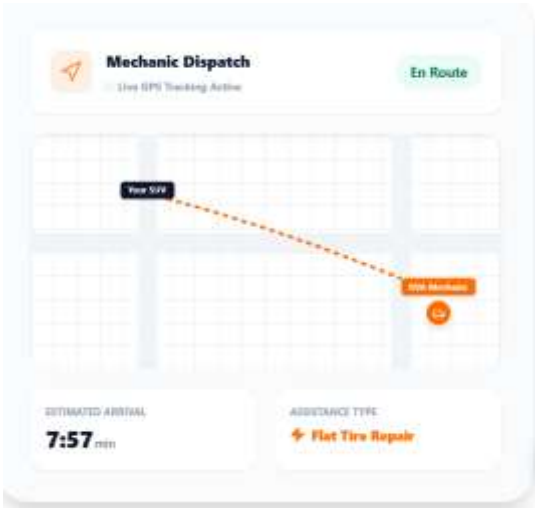
8.6 Nearby Mechanics View

A list of nearby mechanics sorted by distance, showing each mechanic's business name, rating (star display), distance in km, estimated arrival time, and pricing for the selected service type. Each card has a 'Book' button to create a service request.



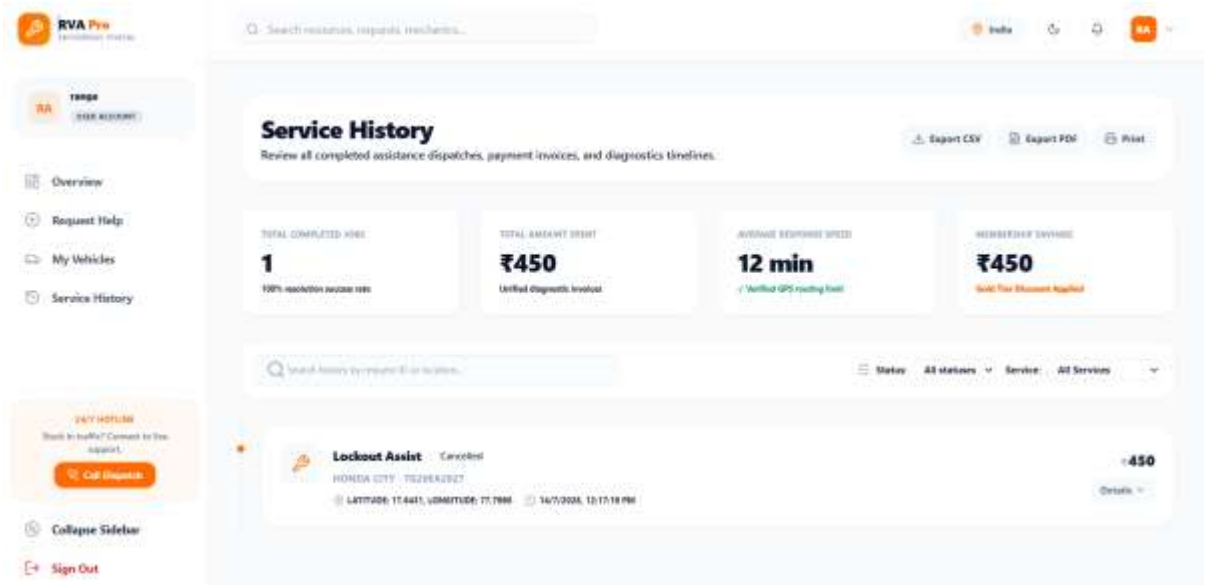
8.7 Live Service Tracking

A real-time tracking view showing the current status of an active service request with a visual step indicator (pending → accepted → en route → arrived → in progress → completed). It displays the assigned mechanic's details, estimated arrival time, and the full status history timeline.



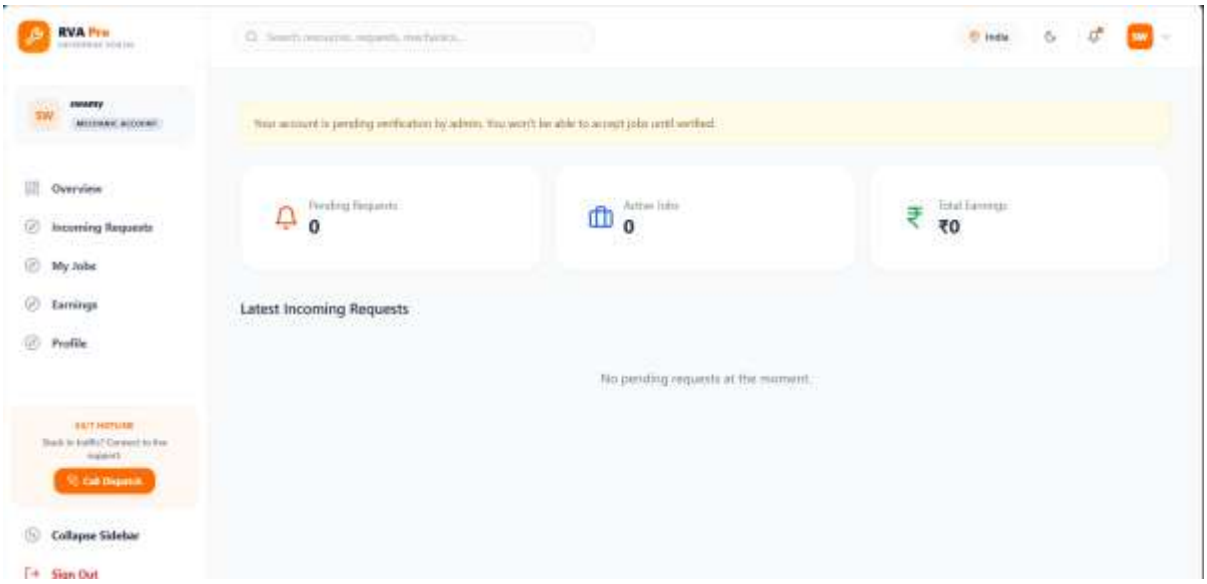
8.8 Service History

A paginated list of past service requests showing service type, date, status badge (color-coded), mechanic name, vehicle used, and price. Each record can be expanded to view full details. Completed services show the rating option if not yet rated.



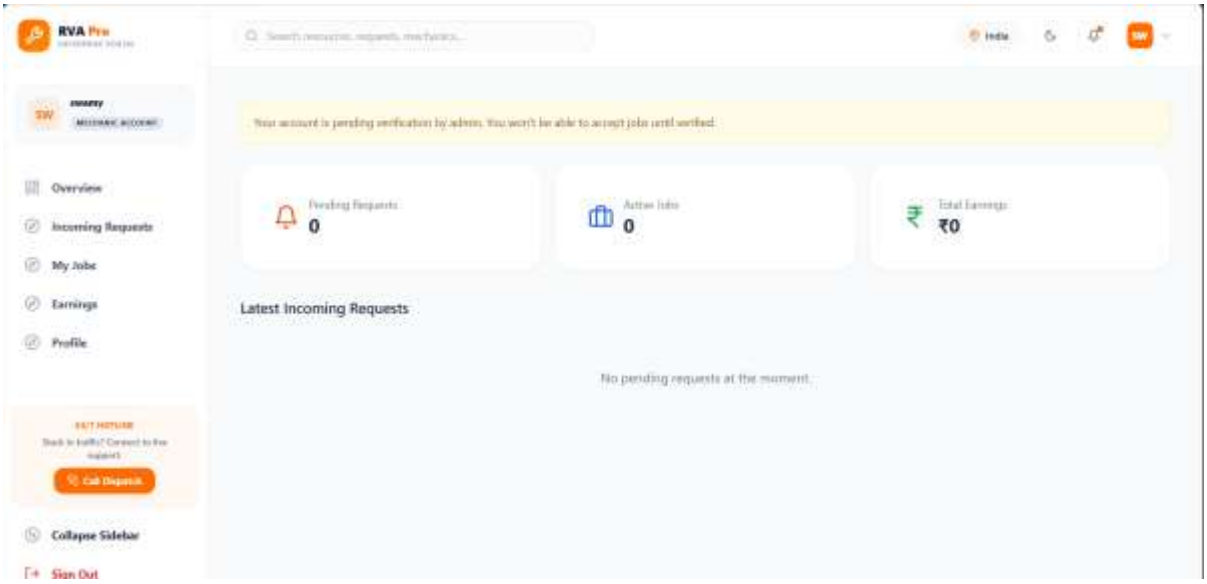
8.9 Mechanic Dashboard

The mechanic dashboard shows active job stats, pending request count, and recent job cards. It includes quick links to view pending requests, update profile/pricing, check earnings, and update availability status.



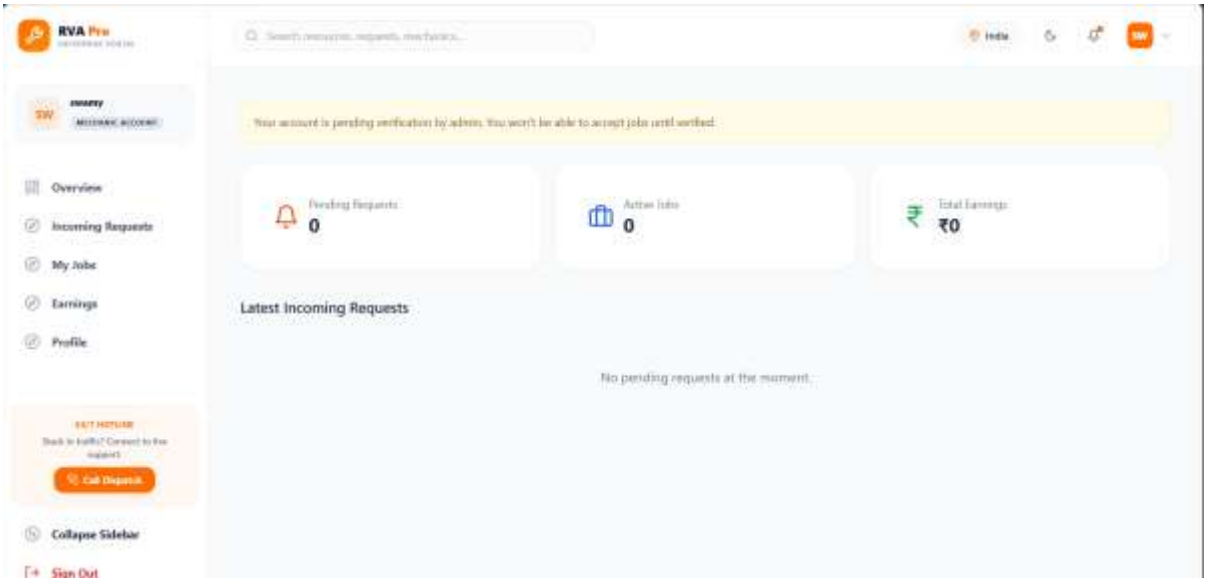
8.10 Mechanic Pending Requests

A list of all pending service requests from users, showing user name, vehicle details, service type, breakdown location address, and distance from the mechanic. Each card has an 'Accept' button to take on the job.



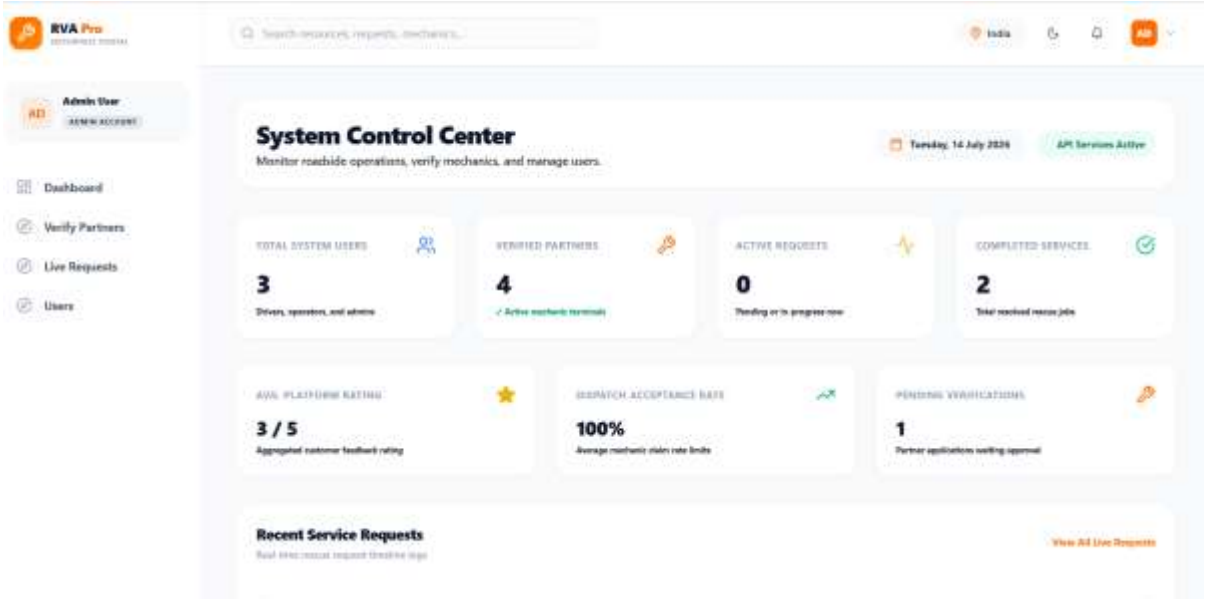
8.11 Mechanic Earnings

An earnings dashboard displaying total earnings, total completed jobs, and a list of recent completed jobs with individual earnings per job.



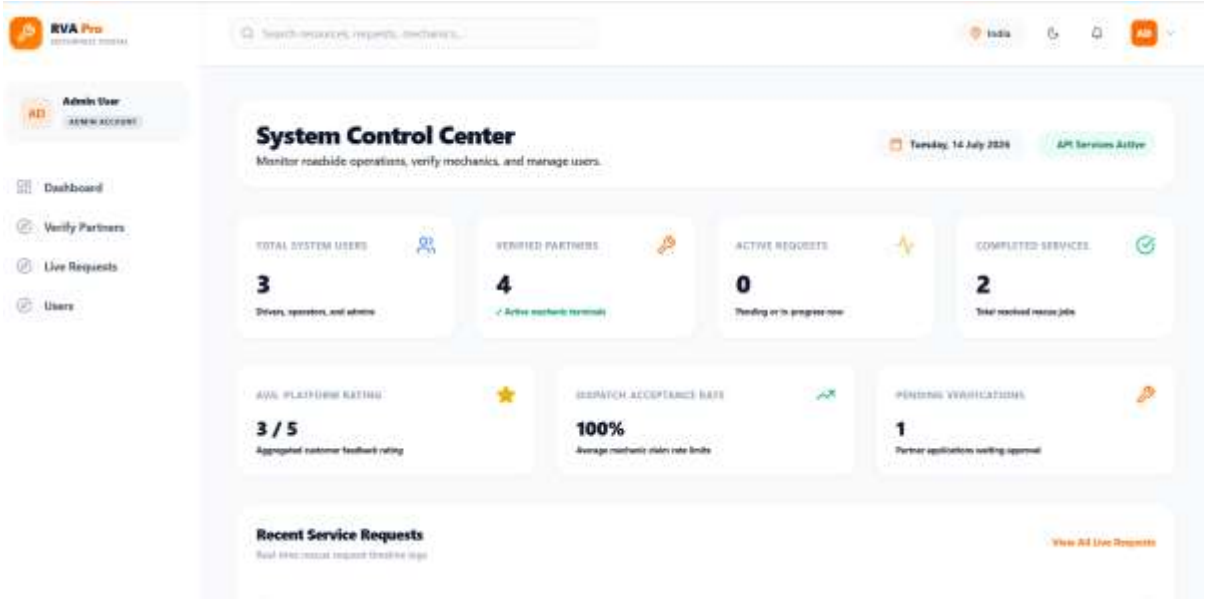
8.12 Admin KPI Dashboard

A comprehensive admin dashboard with KPI cards showing total users, verified mechanics, pending mechanic applications, active service requests, completed requests, average rating, and partner acceptance rate. Below the KPIs, a table lists the 10 most recent service requests.



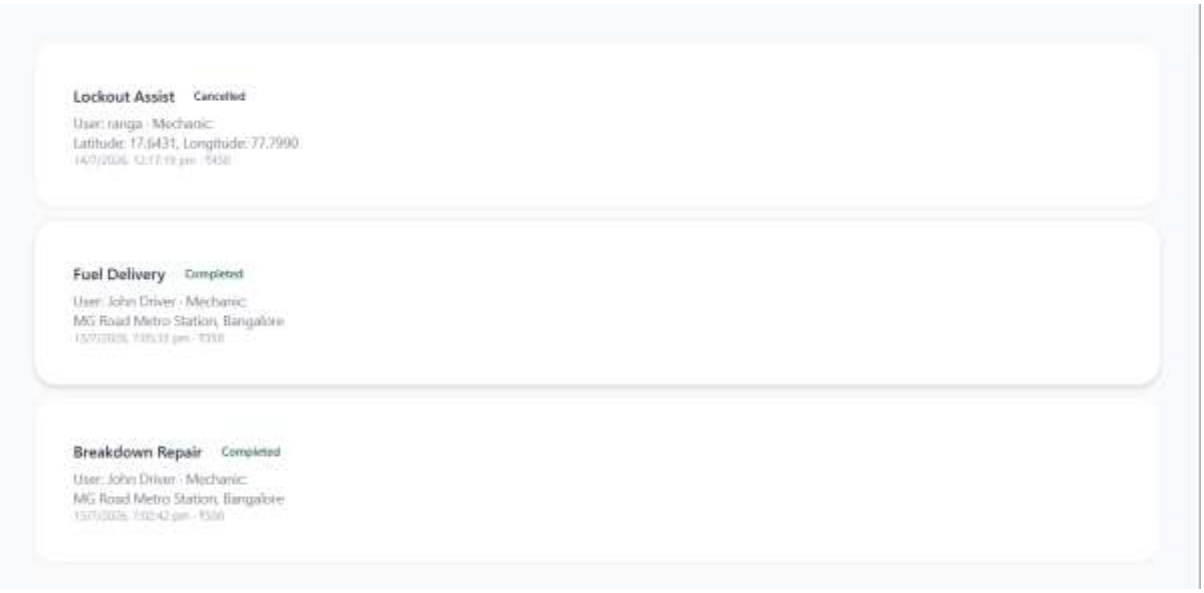
8.13 Admin Partner Verification

A list of pending mechanic applications with details including business name, license number, contact information, and experience. Each application has 'Approve' and 'Reject' buttons for admin action.



8.14 Admin Request Monitoring

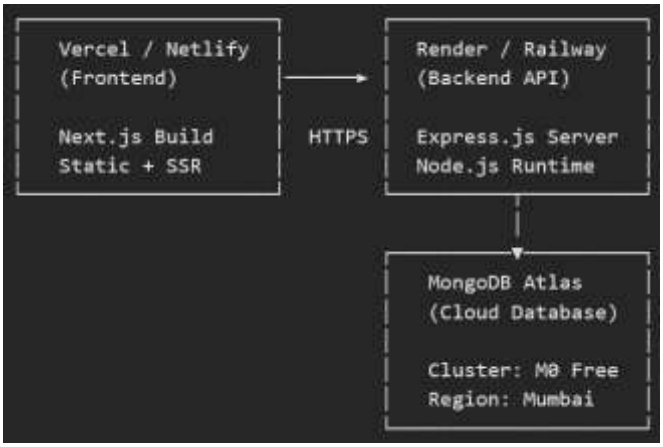
A full list of all service requests on the platform with filtering and status information. Admins can view details and resolve/cancel disputed requests with admin notes.



Chapter 9: Deployment

9.1 Deployment Architecture

Figure 9: Deployment Architecture



9.2 Deployment Steps

9.2.1 Backend Deployment (Render)

1. Push code to GitHub repository
2. Create a new Web Service on Render
3. Connect the GitHub repository

4. Set the build command: npm install --prefix backend
5. Set the start command: node server.js
6. Configure environment variables: MONGODB_URI, JWT_SECRET, NODE_ENV=production
7. Deploy and verify the /api/health endpoint returns status: ok

9.2.2 Frontend Deployment (Vercel)

8. Push code to GitHub repository
9. Import the repository in Vercel
10. Set the root directory to 'frontend'
11. Configure the NEXT_PUBLIC_API_URL environment variable with the deployed backend URL
12. Deploy and verify the application loads correctly

9.2.3 Database Setup (MongoDB Atlas)

13. Create a MongoDB Atlas account and cluster (M0 free tier)
14. Create a database user with read/write permissions
15. Whitelist the backend server's IP address (or 0.0.0.0/0 for development)
16. Copy the connection string to the backend's MONGODB_URI environment variable
17. Run the seed script to populate demo data: npm run seed

9.3 Deployment Checklist

Step	Description	Status
1	Set production JWT_SECRET (strong, unique value)	Required
2	Configure MongoDB Atlas connection string	Required
3	Deploy backend to Render/Railway/AWS	Required
4	Set NEXT_PUBLIC_API_URL on frontend	Required
5	Deploy frontend to Vercel/Netlify	Required
6	Enable CORS for production domain on backend	Required
7	Run seed script for initial demo data	Optional
8	Configure custom domain names	Optional
9	Set up monitoring and logging	Recommended

Chapter 10: Conclusion and Future Scope

10.1 Conclusion

The Roadside Vehicle Assistance & Mechanic Booking Platform has been successfully designed, developed, and tested as a comprehensive full-stack web application. The platform effectively addresses the real-world challenges faced by vehicle owners during breakdowns by providing an integrated digital solution that connects them with verified nearby mechanics and towing service providers.

The project demonstrates proficiency in modern web development technologies including Next.js 14 with React and TypeScript for the frontend, Node.js with Express.js for the backend REST API, and MongoDB with Mongoose for data persistence. Key technical achievements include the implementation of geospatial queries using the Haversine formula for nearby mechanic discovery, JWT-based authentication with role-based access control, a comprehensive service request lifecycle management system, and an admin dashboard with real-time KPI monitoring.

The platform supports three distinct user roles — vehicle owners, mechanics/towing partners, and administrators — each with a tailored dashboard experience. All 19 functional requirements identified in the analysis phase have been successfully implemented and tested. The application is responsive, production-ready, and deployable on modern cloud infrastructure.

This project has provided valuable hands-on experience in full-stack web development, database design, API architecture, authentication systems, geospatial computing, and modern UI/UX design principles.

10.2 Key Performance Indicators

KPI Metric	Description	Implementation
Total Service Requests	Count of all service requests on the platform	MongoDB countDocuments aggregation
Average Response Time	Mean ETA across all accepted requests	Computed from estimatedArrivalMinutes field
Service Completion Rate	Percentage of requests that reach "completed" status	Completed / Total requests ratio
User Satisfaction Rating	Average rating (1–5 stars) across all rated services	MongoDB $avg aggregation on rating field
Partner Acceptance Rate	Percentage of requests accepted by mechanics	Accepted / Total requests ratio

10.3 Future Enhancements (Phase 2+)

- Online Payment Integration: Integrate payment gateways (Razorpay, Stripe) for cashless transactions with automatic invoicing

- Mobile Application: Develop native iOS and Android apps with SOS button for one-tap emergency assistance
- Real-time Communication: Implement WebSocket-based real-time updates replacing polling, with push notifications
- AI-powered ETA: Use machine learning models trained on historical data for more accurate ETA predictions
- Insurance Integration: Partner with insurance companies for direct claim processing through the platform
- Spare Parts Ordering: Add spare parts catalog and ordering capability for mechanics
- Multi-language Support: Implement internationalization (i18n) for regional language support
- Advanced Analytics: Implement detailed analytics dashboards with charts, graphs, and predictive insights
- Driver Safety Features: SOS button, emergency contacts, breakdown history heatmaps
- Subscription Plans: Introduce monthly/annual membership plans for frequent users with discounted rates

10.4 Lessons Learned

- Full-stack architecture design with clear separation of concerns improves maintainability
- GeoJSON and 2dsphere indexes in MongoDB are powerful for location-based features
- JWT-based stateless authentication simplifies horizontal scaling
- TypeScript significantly reduces runtime errors and improves developer experience
- Role-based access control should be enforced on both frontend (routing) and backend (middleware)
- Polling-based updates are simpler to implement but WebSockets are more efficient for real-time features
- Seed data is essential for testing and demo purposes during development

Chapter 11: Bibliography

[1] Next.js Documentation — https://nextjs.org/docs — Vercel Inc., 2024.

[2] React Documentation — https://react.dev — Meta Platforms Inc., 2024.

[3] Express.js Documentation — https://expressjs.com — OpenJS Foundation, 2024.

[4] MongoDB Documentation — https://www.mongodb.com/docs — MongoDB Inc., 2024.

[5] Mongoose ODM Documentation — https://mongoosejs.com/docs — Automattic Inc., 2024.

[6] JSON Web Tokens (JWT) — https://jwt.io — Auth0 Inc., 2024.

[7] Tailwind CSS Documentation — https://tailwindcss.com/docs — Tailwind Labs Inc., 2024.

[8] Node.js Documentation — https://nodejs.org/docs — OpenJS Foundation, 2024.

[9] GeoJSON Specification (RFC 7946) — https://geojson.org — Internet Engineering Task Force, 2016.

[10] Haversine Formula — Sinnott, R.W., 'Virtues of the Haversine,' Sky and Telescope, Vol. 68, No. 2, p. 159, 1984.

[11] REST API Design Best Practices — https://restfulapi.net — REST API Tutorial, 2024.

[12] bcrypt.js — https://github.com/dcodeIO/bcrypt.js — Daniel Wirtz, 2024.

[13] Lucide Icons — https://lucide.dev — Lucide Contributors, 2024.

[14] TypeScript Documentation — https://www.typescriptlang.org/docs — Microsoft, 2024.

[15] MongoDB Atlas — https://www.mongodb.com/atlas — MongoDB Inc., 2024.

[16] Vercel Deployment — https://vercel.com/docs — Vercel Inc., 2024.

[17] Render Deployment — https://render.com/docs — Render Inc., 2024.

Appendix A: Setup and Installation Guide

A.1 Prerequisites

- Node.js 18 or higher (https://nodejs.org)
- MongoDB (local installation or MongoDB Atlas cloud account)
- Git for version control
- A modern web browser (Chrome, Firefox, Edge)
- Code editor (VS Code recommended)

A.2 Backend Setup

Terminal Commands — Backend

```
# Navigate to backend directory
cd backend

# Install dependencies
npm install

# Copy environment template
cp .env.example .env

# Edit .env with your MongoDB URI and JWT secret
# PORT=5000
# MONGODB_URI=mongodb://localhost:27017/roadside_assistance
# JWT_SECRET=your_strong_secret_key

# Load demo/seed data
npm run seed

# Start the development server
npm run dev
# Server starts at http://localhost:5000
```

A.3 Frontend Setup

Terminal Commands — Frontend

```
# Navigate to frontend directory
cd frontend

# Install dependencies
npm install

# Copy environment template
cp .env.local.example .env.local

# Edit .env.local
# NEXT_PUBLIC_API_URL=http://localhost:5000/api
```

```
# Start the development server
npm run dev
# Application starts at http://localhost:3000
```

A.4 Accessing the Application

Open a web browser and navigate to http://localhost:3000. Use the demo accounts listed in Chapter 7, Section 7.5 to log in with different roles and explore the platform features.